Online EFD

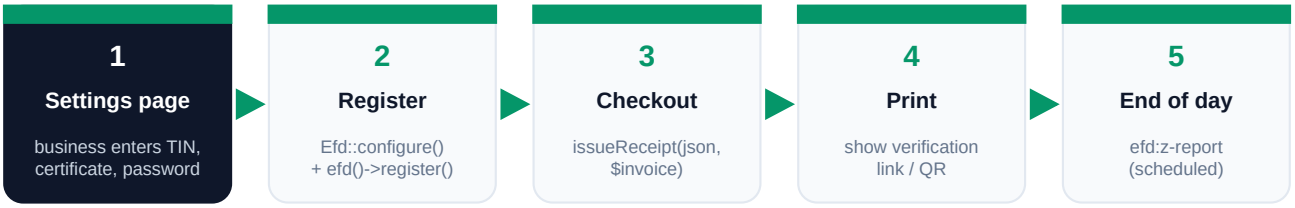
Developer Guide

Issue TRA-compliant fiscal receipts from salesmanager (or any Laravel app). Each business enters its own TRA details once on its settings page; your code then sends a receipt as JSON.

Version **0.4.2** | Updated **2026-10-10** | Composer: `online-efd/online-efd`

1. How it works

The package does the hard TRA work (certificate signing, registration, tokens, receipt XML, counters, retries, Z-reports). You only build two things in salesmanager: a **settings page** and one call at **checkout**.



- **Multi-tenant:** every salesmanager `Business` has its own TIN, certificate, TRA login and counters.
- **JSON in, JSON out:** send an array or JSON string, get a receipt record that serialises to JSON.
- **Safe:** certificate, passwords and tokens are encrypted in the database and never returned by the package.
- **Forgiving:** if TRA is down the receipt is kept and can be retried with the same numbers. The same invoice never gets two receipts.

Note. Keep APP_KEY stable. Stored certificates are encrypted with it and become unreadable if it changes.

2. Install (5 minutes)

1 Add the package

While developing locally, point salesmanager's `composer.json` at the folder:

```
"repositories": [
  { "type": "path", "url": "../vfd-master/online_efd", "options": { "symlink": true } }
]

composer require online-efd/online-efd:@dev
php artisan migrate        # creates efd_profiles, efd_receipts, efd_z_reports
```

2 Mark the tenant model

In salesmanager the tenant is `Business` and the sale is `Invoice`.

```
// app/Models/Business.php
use OnlineEfd\Concerns\HasEfdProfile;

class Business extends Model
{
    use HasEfdProfile;      // gives $business->efd() and $business->hasEfd()
    // ...
}
```

Optional: `php artisan vendor:publish --tag=efd-config` to change endpoints, timezone or queue settings.

3. Real usage flow in salesmanager

Adapt model, route and relation names to your code. Everything below uses the package's real functions.

1 Settings page: the business saves its TRA details

Four fields: TIN, certificate key (e.g. `10TXXXXX73`), the `.pfx` file and its password. These come from TRA when the business is approved for VFD.

```
// routes/web.php
Route::middleware('auth')->group(function () {
    Route::get('/settings/efd',          [EfdSettingsController::class, 'edit'])->name('efd.edit');
    Route::post('/settings/efd',         [EfdSettingsController::class, 'update'])->name('efd.update');
    Route::post('/settings/efd/register', [EfdSettingsController::class, 'register'])->name('efd.register');
});

// app/Http/Controllers/EfdSettingsController.php
use OnlineEfd\Exceptions\EfdException;
use OnlineEfd\Facades\Efd;

class EfdSettingsController extends Controller
{
    private function business(Request $request): Business
    {
        // Always resolve from the logged-in user, never from a request id.
        return $request->user()->business;          // however salesmanager resolves it
    }

    public function edit(Request $request)
    {
        return view('settings.efd', ['efd' => Efd::status($this->business($request))]);
    }

    public function update(Request $request)
    {
        $data = $request->validate([
            'tin'                => ['required', 'string', 'max:20'],
            'cert_key'           => ['required', 'string', 'max:50'],
            'certificate'        => ['nullable', 'file', 'max:256'],
            'certificate_password' => ['nullable', 'string'],
        ]);

        try {
            Efd::configure($this->business($request), $data);   // opens the .pfx to verify it
        } catch (EfdException $e) {
            return back()->withErrors(['efd' => $e->getMessage()])->withInput();
        }

        return back()->with('status', 'EFD settings saved.');
    }

    public function register(Request $request)
    {
        try {
            $this->business($request)->efd()->register();   // talks to TRA
        } catch (EfdException $e) {
            return back()->withErrors(['efd' => $e->getMessage()]);
        }

        return back()->with('status', 'Registered with TRA.');
    }
}

{{-- resources/views/settings/efd.blade.php --}}
<form method="POST" action="{{ route('efd.update') }}" enctype="multipart/form-data">
```

```
    @csrf
    <input name="tin"      value="{{ old('tin', $efd['tin']) }}"           placeholder="TIN">
    <input name="cert_key" value="{{ old('cert_key', $efd['cert_key']) }}" placeholder="Certificate key">
    <input type="file"     name="certificate" accept=".pfx,.p12">
    <input type="password" name="certificate_password" placeholder="Certificate password">
    <button>Save</button>
</form>

@if ($efd['has_certificate'])
    Certificate expires {{ \Carbon\Carbon::parse($efd['certificate_expires_at'])->toFormattedDateString() }}
@endif

@if ($efd['registered'])
    Registered: {{ $efd['taxpayer']['name'] }} (VRN {{ $efd['taxpayer']['vrn'] }})
@elseif ($efd['has_certificate'])
    <form method="POST" action="{{ route('efd.register') }}">@csrf<button>Register with TRA</button></form>
@endif
```

Tip. `Efd::status()` is safe to send to the page or an API: it never contains the certificate, passwords or tokens. Fields you leave empty on later saves are kept as they are.

2 Checkout: issue the receipt when an invoice is paid

Map the invoice to the receipt JSON and call `issueReceipt()`. Passing the invoice as the second argument means calling it twice for the same invoice returns the same receipt.

```
// app/Services/FiscalReceiptService.php
use OnlineEfd\Exceptions\EfdException;
use OnlineEfd\Facades\Efd;

class FiscalReceiptService
{
    public function issueFor(Invoice $invoice): ?\OnlineEfd\Models\EfdReceipt
    {
        $business = Business::findOrFail($invoice->business_id);

        if (! Efd::isReady($business)) {
            return null;                              // EFD not set up: sell as usual
        }

        $payload = [
            'items' => $invoice->invoiceProducts->map(fn ($line) => [
                'id'          => (string) $line->product_id,
                'description' => $line->product->name,
                'quantity'    => $line->quantity,
                'unit_price'  => $line->sale_price,   // must be the final, VAT-inclusive price
                'tax_code'    => 1,                   // 1 = 18% VAT, 5 = exempt (see section 4)
            ])->all(),
            'payment_type' => 'CASH',                 // CASH | CHEQUE | CCARD | EMONEY | INVOICE
            'customer'     => $invoice->client ? [
                'name'  => $invoice->client->name,
                'phone' => $invoice->client->phone,
            ] : null,
        ];

        try {
            return $business->efd()->issueReceipt($payload, $invoice);
        } catch (EfdException $e) {
            report($e);        // receipt is saved as "failed" and can be retried (step 4)
            return null;       // do not block the sale because TRA is unreachable
        }
    }
}

// where the invoice is finalised
$receipt = app(FiscalReceiptService::class)->issueFor($invoice);
```

```
$receipt?->verification_url;   // https://verify.tra.go.tz/...  (print it, or turn it into a QR)
$receipt?->rctnum;             // fiscal receipt number to print on the invoice
```

Prefer not to wait for TRA at checkout? Use the queue instead. The receipt is reserved immediately and posted in the background, with automatic retries:

```
$business->efd()->issueReceiptLater($payload, $invoice);   // needs a running queue worker
```

3 Same thing over JSON (API or mobile/POS clients)

The receipt payload is plain JSON, so it works the same from an API endpoint:

```
// routes/api.php
Route::post('/invoices/{invoice}/efd', function (Request $request, Invoice $invoice) {
    $business = $request->user()->business;
    abort_unless($invoice->business_id === $business->id, 403);

    return response()->json(
        $business->efd()->issueReceipt($request->all(), $invoice)    // also accepts a JSON string
    );
});
```

```
curl -X POST https://app.example.com/api/invoices/1042/efd \
  -H "Authorization: Bearer <token>" -H "Content-Type: application/json" \
  -d '{"items":[{"description":"Soda 500ml","quantity":3,"unit_price":1500},
                {"description":"Bread","quantity":1,"unit_price":2000,"tax_code":5}],
       "payment_type":"CASH"}'
```

```
{ "id": 87, "rctnum": 14, "gc": 14, "dc": 3, "znum": "20261010",
  "total_excl": "5813.56", "total_incl": "6500.00", "payment_type": "CASH",
  "status": "posted", "ack_code": "0", "ack_message": "Success",
  "verification_url": "https://verify.tra.go.tz/A1B2C314_143055",
  "posted_at": "2026-10-10T14:30:56.000000Z", "items": [ ... ], "vat_totals": { "A": { ... } } }
```

4 When TRA was unreachable: retry failed receipts

A failed receipt keeps its receipt number and counters, so retrying never leaves a gap. The package ships a command that re-sends every real receipt TRA has not accepted yet (test receipts are never retried). Schedule it every few minutes, and it also clears the way for the Z-report:

```
// routes/console.php (Laravel 11+)  or  app/Console/Kernel.php::schedule() (Laravel 10)
Schedule::command('efd:retry')->everyFiveMinutes()->withoutOverlapping();

// by hand
php artisan efd:retry --limit=50 --minutes=2
```

5 End of day: Z-report

TRA expects one Z-report per business per day, after the last sale. The command posts it for every registered business. It refuses to run for a business that still has unposted receipts that day, which is why step 4 matters.

```
Schedule::command('efd:z-report')->dailyAt('23:55')->timezone('Africa/Dar_es_Salaam');

// or manually for one business / one day
$business->efd()->postZReport(Carbon::parse('2026-10-09'));
```

6 Try it first from tinker (no UI needed)

```
php artisan tinker
>>> $b = App\Models\Business::first();
>>> OnlineEfd\Facades\Efd::configure($b, [
...   'tin' => '150000091', 'cert_key' => '10TXXXXX73',
...   'certificate' => 'D:/certs/vfd.pfx',            // a file path works too
...   'certificate_password' => '******',
... ]);
>>> $b->efd()->register();
>>> $b->efd()->testConnection();                       // proves the TRA login works
>>> $b->efd()->issueReceipt(['items' => [['description' => 'Test', 'quantity' => 1, 'unit_price' => 1000]]]);
```

4. Reference

Functions

Call	What it does
`Efd::configure($owner, array)`	Save tin, cert_key, certificate (file / path / bytes), certificate_password. Verifies the .pfx. Partial updates allowed.
`Efd::status($owner)`	JSON-safe summary for the settings page. No secrets.
`Efd::isReady($owner)`	true when configured, registered and able to issue receipts.
`Efd::removeCertificate($owner)`	Delete the stored certificate.
`$owner->efd()->register()`	Register with TRA; stores the credentials TRA returns.
`$owner->efd()->testConnection()`	Check the stored TRA login works.
`$owner->efd()->issueReceipt($json, $sale)`	Create and post a receipt now. Returns the receipt record.
`$owner->efd()->issueReceiptLater($json, $sale)`	Reserve it now, post from the queue.
`$owner->efd()->issueTestReceipt($json)`	Send a test receipt: real to TRA, flagged as a test, kept out of Z-reports.
`Efd::forProfile($p)->retry($receipt)`	Re-send a failed/pending receipt.
`$owner->efd()->postZReport(?Carbon)`	Post the day's Z-report (default: today).

Receipt JSON

```
{
  "items": [                                  // required, at least one
    { "description": "Soda 500ml",            // required
      "quantity": 3,                          // required
      "unit_price": 1500,                     // required, VAT-INCLUSIVE
      "tax_code": 1,                          // optional, default 1
      "id": "SKU-1" }                         // optional, default = line number
  ],
  "payment_type": "CASH",                     // optional, default CASH
  "customer": { "name": "Jane", "phone": "255700000000", "id_type": 5, "id": "1234" },
  "issued_at": "2026-10-10 14:30:00"          // optional, default now
}
```

Field	Allowed values
`tax_code`	1 = 18% VAT (default) | 2 = special rate | 3 = zero rated | 4 = special relief | 5 = exempt

Field	Allowed values
`payment_type`	CASH | CHEQUE | CCARD (credit card) | EMONEY (mobile money) | INVOICE (credit sale)
`customer.id_type`	1 TIN | 2 driving licence | 3 voter | 4 passport | 5 national ID | 6 none (default)

Bad input throws `InvalidArgumentException` naming the field (e.g. *items.0.unit_price is required*). TRA, network and certificate problems throw `OnlineEfd\Exceptions\EfdException` with a message that is safe to show to users.

Efd::status() result

```
{ "configured": true, "tin": "150000091", "cert_key": "10TXXXXX73",
  "has_certificate": true, "certificate_expires_at": "2027-03-01T00:00:00+00:00",
  "registered": true, "ready": true,
  "taxpayer": { "name": "ACME LTD", "vrn": "...", "regid": "TZ...", "tax_office": "..." } }
```

Receipt statuses and events

- **pending**: reserved, not yet accepted. **posted**: accepted by TRA. **failed**: rejected or unreachable, retry it.
- Listen for `OnlineEfd\Events\ReceiptPosted` and `ReceiptFailed` to notify users or store the link.

Config (php artisan vendor:publish --tag=efd-config)

Key	Default / purpose
`timezone`	Africa/Dar_es_Salaam. Receipt time, Z-number and the daily counter reset use it.
`api.base_url`	https://virtual.tra.go.tz/efdmsRctApi (env EFD_BASE_URL)
`api.verify_ssl`	true (env EFD_VERIFY_SSL). Fix the server's CA bundle rather than turning it off.
`tax_rates`	Fallback VAT %. The rates TRA returns at registration take precedence.
`queue.*`	connection, queue, tries (5) and backoff (30/120/600 s) for issueReceiptLater().

5. How salesmanager uses it

salesmanager is multi-tenant and multi-business, with the subscription held by the account owner. The package is wired in like this (code in `app/Services/BusinessEfd.php` and `app/Http/Controllers/API/EfdController.php` of salesmanager):

- **Paid add-on, also a package feature.** EFD is the package feature `efd` and an add-on the platform admin prices separately (same billing periods as plans). A business can use EFD when its plan includes the feature or its account has an active add-on. Unlike other features this fails closed: no plan, no EFD.
- **Shared or own details, switchable any time.** Each business is set to *off*, *shared* or *own*. *Shared* keeps one profile on the account owner, so the TIN, certificate and receipt counter are used by every business the owner ticks. *Own* keeps a profile on the business itself. Switching never deletes either profile.
- **Why shared means one profile.** TRA numbers receipts per registration, so businesses on the same TIN and certificate must share one counter. They cannot each keep their own.

- **Who can change what.** Anyone with the general-settings permission manages a business's own details. Shared details and the sharing list are the account owner's only.
- **Receipts when an invoice is paid.** With the per-business switch on, an invoice that becomes fully paid is turned into a receipt after the sale is saved (a queued job, never part of the checkout request). VAT, discount and returns are folded into the line prices and transport becomes its own line, so the receipt equals what the customer paid to the cent. A failure never affects the sale: it is recorded and re-sent by `efd:retry`.
- **Number and QR on printed documents.** Once TRA has accepted a receipt, the invoice page, the POS slip and the invoice PDF show the receipt number, the verification code (`RCTVNUM` followed by the global counter) and a QR of `verification_url`. The POS waits a few seconds after a fully paid sale so they are on the slip.
- **Test area per business.** A simple receipt form posts through `issueTestReceipt()`; every attempt (accepted, rejected, or not sent) is kept in `efd_test_runs` with what was sent and TRA's reply.

Note. Test receipts are real requests to TRA and use real receipt numbers. We flag them and leave them out of our Z-report, but TRA still records them. Use small amounts, and ask TRA whether a separate test environment is available.

6. Rules worth knowing

- **Prices are VAT-inclusive.** The package splits them: net = price / 1.18, VAT = price - net. Lines always add up exactly.
- **One receipt per invoice.** Pass the invoice as the second argument and repeated calls return the same receipt.
- **Numbers never skip.** Each receipt gets the next global and daily counter inside a database lock; a failed receipt keeps its numbers for the retry.
- **Z-report is all-or-nothing for the day.** It is refused while any real receipt of that day is not accepted by TRA. Test receipts are ignored.
- **Changing TIN or certificate key** on a registered business clears its registration; register again.
- **Authorization is yours.** The package trusts the owner model you pass it. Resolve the business from the logged-in user.

What is stored

Table	Contents
`efd_profiles`	One per business: TIN, certificate key, encrypted .pfx + password, TRA login, taxpayer details, counters, cumulative gross, cached token (encrypted).
`efd_receipts`	One per receipt: numbers, amounts, VAT split, status, TRA reply, signed request/response XML, verification link, link to your invoice.
`efd_z_reports`	One per business per day: totals, status, TRA reply.

Under the hood (for debugging)

Step	TRA call
Register	POST /api/vfdRegReq with signed REGDATA (TIN + certificate key). Returns the TRA login and taxpayer details.
Token	POST /vfdtoken (username, password). Cached until 30 s before it expires.
Receipt	POST /api/efdmsRctInfo, signed RCT XML, bearer token, headers Cert-Serial, Routing-Key, Client.
Z-report	POST /api/efdmszreport, signed ZREPORT XML.

Signing is SHA1withRSA with the business's own `.pfx`, as TRA requires.

7. Troubleshooting

Message / symptom	Fix
Certificate error: could not open the .pfx	Wrong password, or an old-format .pfx on OpenSSL 3. Re-export: openssl pkcs12 -legacy -in old.pfx -nodes -out tmp.pem, then openssl pkcs12 -export -in tmp.pem -out new.pfx.
TRA rejected the request (code N)	The text after the code comes from TRA. Check TIN, certificate key, certificate expiry and the server clock.
Could not reach TRA	Network or TRA outage. The receipt is saved as failed; efd:retry or issueReceiptLater() will send it.
cURL error 60 / SSL certificate problem	Update the server's CA bundle (curl.cainfo in php.ini). Do not disable verify_ssl.
Cannot post Z-report: receipts not accepted	Retry the failed receipts for that day, then post again.
Stored certificate suddenly unreadable	APP_KEY was changed. Restore it, or upload the certificate again.

Known limitations

- **Not yet verified against live TRA:** the counter rule (next number = registration GC + 1) and the verification-link format. Run one real registration and receipt with your test certificate before go-live.
- No discounts, voids, credit notes or corrections yet (sent as zero).
- TRA's response signature is not verified. One EFD profile per business.

Tests

```
cd online_efd && composer install && vendor/bin/phpunit      # 20 tests, TRA is faked
```

8. Changelog

0.4.2 2026-10-10

- Sample data in the README, guide and tests replaced with placeholders (no real TIN, certificate key, registration or contact details).

0.4.1 2026-10-10

- salesmanager prints the receipt number, verification code and QR on the invoice page, the POS slip and the invoice PDF.

0.4.0 2026-10-10

- New command efd:retry: re-sends real (non-test) receipts TRA has not accepted, with their original numbers. Schedule it every few minutes.
- salesmanager now issues a receipt automatically when an invoice is paid in full (per-business switch, off by default).

0.3.0 2026-10-10

- Test receipts: issueTestReceipt() sends a receipt flagged is_test. TRA still receives it and it uses the next counters, but it is kept out of Z-reports and never blocks one.
- EfdException now carries the stored receipt ($e->receipt) so callers can log rejected receipts.
- Integrated into salesmanager: EFD add-on, shared/own details per business, settings page and per-business test area.

0.2.1 2026-10-10

- Documentation redesigned: salesmanager theme and logo, shorter, step-by-step usage flow with Laravel and PHP samples.

0.2.0 2026-10-10

- Simplified: removed Livewire, Blade views and Gate-based authorization. The package is UI-free.
- Added Efd::configure(), Efd::status(), Efd::removeCertificate().
- Receipts can be issued from a JSON string or array, with field validation.
- 18 tests passing.

0.1.0 2026-10-10

- Initial package: profiles, registration, token cache, receipts, Z-reports, queue job, events, Artisan command.

Maintaining this guide: edit docs/build_documentation.py, add a changelog entry, bump VERSION, run python docs/build_documentation.py.